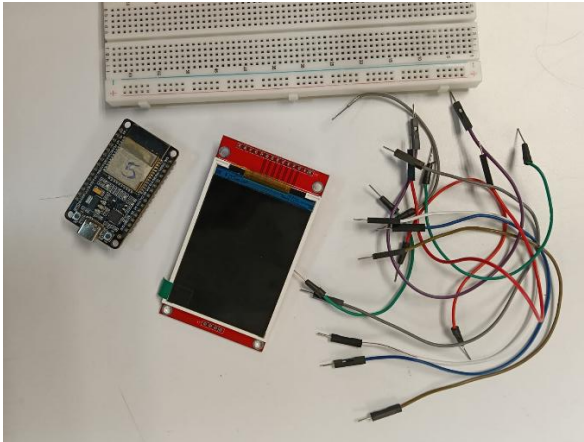


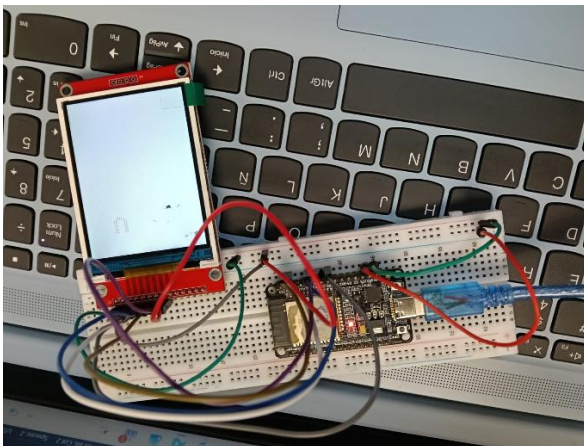
Laboratorio 8 SPI en Acción: Comunicación con Pantalla TFT ILI9341

Materiales:

- ESP32 DevKit o Arduino UNO/Mega (ESP32 recomendado por mayor velocidad SPI)
- Pantalla TFT 2.4 pulgadas ILI9341 (240x320) con interfaz SPI
- Jumpers macho-hembra (8 piezas mínimo)
- Sensor BMP280 o DHT22 (para mostrar datos en pantalla)
- Arduino IDE 2.x con bibliotecas: Adafruit ILI9341, Adafruit GFX instaladas



Circuito armado



Preguntas de reflexión y reporte

¿Cuántos bytes de datos se necesitan para actualizar un solo pixel en la pantalla ILI9341? ¿Y para actualizar toda la pantalla (240x320)?

La pantalla ILI9341 usa formato de color RGB565, lo que significa que cada píxel ocupa 16 bits, es decir, 2 bytes.

1 píxel = 2 bytes

La pantalla tiene una resolución de 240 x 320 píxeles, esto quiere decir que

$240 \times 320 = 76,800$ píxeles

$76,800 \times 2 = 153,600$ bytes

Para actualizar toda la pantalla se necesitan 153,600 bytes.

A 40 MHz de SPI, ¿cuánto tiempo mínimo (teórico) se necesita para enviar todos los datos de un frame completo? ¿Por qué en la práctica es mayor?

A 40 MHz, la comunicación SPI envía datos muy rápido. Teóricamente, mandar toda la pantalla tarda aproximadamente 30.7 milisegundos.

Eso significa que se podrían mostrar alrededor de 32 imágenes por segundo.

Pero en la práctica tarda más porque: También se envían comandos, no solo datos, el ESP32 necesita procesar instrucciones, la pantalla tarda un poco en responder, puede haber pequeñas pérdidas por cables o conexiones.

¿Por qué se usa la función fillRect() para borrar el área del número antes de redibujar el nuevo valor, en lugar de usar fillScreen() completo?

Se usa fillRect() porque solo borra una parte pequeña de la pantalla, donde cambia el número de temperatura o humedad.

Esto es mejor porque: Es más rápido, no borra toda la interfaz, evita parpadeos, usa menos datos por SPI, hace que la pantalla responda mejor.

Si se usara fillScreen(), toda la pantalla se borraría y se tendría que dibujar otra vez completa.

Compara la velocidad de actualización de la pantalla cuando el ESP32 opera el SPI a 10 MHz vs 40 MHz. ¿Qué diferencia se observa visualmente?

A 10 MHz:

La pantalla actualiza más lento.

Los cambios tardan más en verse.

Puede notarse parpadeo.

A 40 MHz:

La pantalla responde más rápido.

Los números cambian casi al instante.

Se ve más fluida y suave.

El de 40 MHz el funcionamiento se ve mucho mejor.

¿Cuáles son las ventajas de usar SPI sobre I2C para una pantalla gráfica de alta resolución? ¿Existiría alguna ventaja de usar I2C para este caso?

SPI tiene varias ventajas sobre I2C cuando se usa una pantalla gráfica de alta resolución. La principal es que SPI es mucho más rápido, por lo que puede enviar grandes cantidades de datos en menos tiempo. Esto permite que la pantalla actualice imágenes, texto y animaciones de forma más fluida, ofrece mejor rendimiento en pantallas a color, como la ILI9341, donde se necesita transmitir muchos píxeles constantemente. En cambio, I2C es más lento, por lo que una pantalla de alta resolución tardaría más en actualizarse y podría verse con retraso o parpadeo.

La ventaja de I2C en este caso sería que usa menos pines, ya que solo necesita dos líneas de comunicación SDA y SCL, lo que facilita las conexiones y permite conectar varios dispositivos en el mismo bus.